



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	14 / Tipe Data Set

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Pengenalan dan Mendefinisikan Set

- **Set** adalah salah satu tipe data pada Python yang dapat digunakan untuk menyimpan sekumpulan data yang semuanya unik.
- Biasanya juga dikenal dengan sebutan **himpunan**.
- Sifat-sifat tipe data Set pada **Python** :
 - 1) Isi dari Set = anggota (member).
 - 2) Anggota (member) dari Set harus bersifat **immutable**. Tipe data **immutable** pada **Python** adalah integer, float, string, tuple, dan lain-lain. Sementara list dan dictionary yang bersifat **mutable** tidak dapat dimasukkan ke dalam Set.
 - 3) Tipe data Set sendiri bersifat **mutable**, yang berarti dapat ditambah maupun dikurangi isinya. Maka dari itu, Set tidak dapat dimasukkan ke dalam Set.
- Beberapa cara yang dapat dilakukan untuk mendefinisikan Set, dapat menggunakan notasi `{ }` dan fungsi `set()`.

```
1 # dengan menggunakan { }
2 bil_kelipatan_2 = {2, 4, 6, 8, 10}
3 bil_kelipatan_5 = {5, 10, 15, 20, 25}
4
5 # dengan menggunakan fungsi set()
6 informatika_25 = set('Angela Revaline', 'Cheila Zefanya',
7 | 'Alicia Luna', 'Virjie Gracia')
```

- Untuk mendefinisikan Set kosong dapat menggunakan notasi `{ }`, namun harus menggunakan fungsi `set()`.

```
1 # dengan fungsi set()
2 informatika_25 = set()      #menghasilkan set kosong
3
4 data = {}                  #menghasilkan dictionary kosong
```

2. Pengaksesan Set

- Set tidak memiliki indeks, maka dari itu kita tidak bisa mengakses anggota-anggota dari sebuah Set secara langsung menggunakan indeks.

```
1 nim = {'71251188', '71251189', '71251191', '71251193'}
2 jumlah_nim = len(nim)
3 print(jumlah_nim)      #akan menghasilkan output 4
4
5 #menampilkan isi set satu-persatu
6 for i in nim:
7 |     print(i)
```

Output :

```
4
71251193
71251191
71251189
71251188
```

- Output yang dihasilkan tidak urut seperti deklarasi Set sebelumnya. Hal ini terjadi karena pada Set tidak ada indeks, sehingga tidak ada urutan posisi anggotanya.
- Set merupakan tipe data yang bersifat **mutable**, artinya isinya bisa ditambah ataupun dikurangi.
- Berikut adalah contoh program yang menunjukkan bagaimana cara menambah anggota dalam sebuah Set menggunakan fungsi **add()**.

```
1  #definisikan sebuah set kosong
2  nim = set()
3
4  #menambah nim '71251188'
5  nim.add('71251188')
6
7  #menambah nim '71251189'
8  nim.add('71251189')
9
10 #jumlah anggota dalam set
11 print(len(nim))
12
13 #menambah nim yang sama sekali lagi '71251188'
14 nim.add('71251188')
15
16 #tampilkan semua nim
17 for n in nim:
18     print(n)
```

Output :

```
2
71251188
71251189
```

- Set memiliki mekanisme yang berfungsi untuk mengecek apakah anggota baru yang dimasukkan sudah ada di dalam Set atau belum (**cek duplikasi**). Jika belum ada, maka anggota tersebut bisa masuk ke dalam Set. Namun, jika sudah ada nilai yang sama, maka pemanggilan fungsi **add()** tidak akan menambah dan merubah apapun dalam Set. Pengecekan tersebut sudah ada dalam fungsi **add()**, sehingga kita tidak perlu melakukannya sendiri.

- Terdapat beberapa cara untuk menghapus anggota dalam sebuah Set, yaitu :
- 1) **discard()** = menghapus satu elemen yang disebutkan. (tidak ada error)

```
1 bil_kelipatan_3 = {3, 6, 9, 12, 15}
2
3 bil_kelipatan_3.discard(17)
4 print(bil_kelipatan_3)
```

Output :

```
{3, 6, 9, 12, 15}
```

Fungsi **discard()** tidak akan menghasilkan error jika anggota yang ingin dihapus tidak ada di dalam Set.

- 2) **remove()** = menghapus satu elemen yang disebutkan. (muncul error jika elemen yang dihapus tidak ada)

```
1 bil_kelipatan_3 = {3, 6, 9, 12, 15}
2
3 bil_kelipatan_3.remove(9)
4 print(bil_kelipatan_3)
```

Output :

```
{3, 6, 12, 15}
```

Jika elemen yang ingin dihapus tidak ada :

```
Traceback (most recent call last):
  File "c:\Users\katar\Downloads\minggu14\laprak.py", line 3, in <module>
    bil_kelipatan_3.remove(17)
    ~~~~~^~~~~~
KeyError: 17
```

- 3) **pop()** = mengambil salah satu dan menghapusnya dari set (tidak tentu). (error jika set kosong)

```
1 bil_kelipatan_3 = {3, 6, 9, 12, 15}
2
3 bil = bil_kelipatan_3.pop()
4 print(bil)
5 print(bil_kelipatan_3)
```

Output :

```
3
{6, 9, 12, 15}
```

Fungsi **pop()** akan mengambil salah satu anggota (secara acak) dan mengeluarkannya dari Set. Fungsi **pop()** akan berguna jika kita ingin memproses isi dari Set satu-persatu tanpa memperdulikan urutan / posisi dari setiap anggota di dalam Set.

- 4) **clear()** = menghapus seluruh elemen dalam set. (tidak ada error)

```
1 bil_kelipatan_3 = {3, 6, 9, 12, 15}
2
3 bil_kelipatan_3.clear()
4 print(bil_kelipatan_3)
```

Output :

```
set()
```

- Contoh program yang melakukan operasi penggantian (**replace**) dengan cara menghapus anggota yang mau diubah lalu memasukkan anggota baru yang nilainya sesuai dengan yang diinginkan :

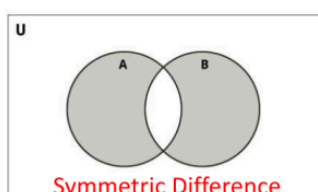
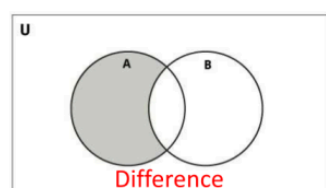
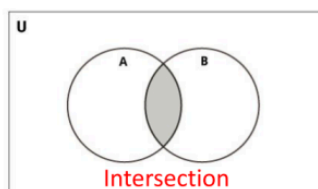
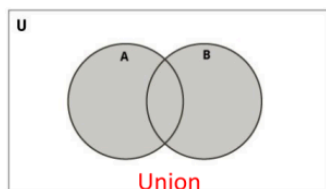
```
1 merk_hp = set(['Samsung', 'Apple', 'Xiaomi', 'Oppo'])
2 print(merk_hp)
3
4 merk_hp.remove('Oppo')
5 merk_hp.add('Infinix')
6 print(merk_hp)
```

Output :

```
{'Samsung', 'Xiaomi', 'Apple', 'Oppo'}
{'Samsung', 'Xiaomi', 'Apple', 'Infinix'}
```

3. Operasi-Operasi pada Set

- Operasi-operasi pada Set adalah operasi-operasi pada himpunan.
➤ Daftar operasi-operasi Set pada **Python** :



1) **Operator Union** = menggabungkan dua Set menjadi satu.

Bisa menggunakan operator `|` maupun fungsi `union()`.

```
1 buah_a = {'apel', 'jeruk', 'durian', 'mangga'}
2 buah_b = {'anggur', 'belimbing', 'nanas', 'alpukat'}
3
4 gabungan = buah_a | buah_b
5 print(gabungan)
```

Output :

```
{'durian', 'anggur', 'apel', 'alpukat', 'jeruk', 'mangga', 'nanas', 'belimbing'}
```

2) **Operator Intersection** = menghasilkan irisan dari dua Set.

Bisa menggunakan operator `&` maupun fungsi `intersection()`.

```
1 buah_a = {'apel', 'jeruk', 'durian', 'mangga'}
2 buah_b = {'anggur', 'belimbing', 'jeruk', 'alpukat'}
3
4 ab = buah_a & buah_b
5 print(ab)
```

Output :

```
{'jeruk'}
```

3) **Operator Difference** = menghasilkan Set baru yang merupakan selisih dari dua Set yang dibandingkan.

Bisa menggunakan operator `-` maupun fungsi `difference()`.

```
1 buah_a = {'apel', 'jeruk', 'durian', 'mangga'}
2 buah_b = {'anggur', 'belimbing', 'jeruk', 'alpukat'}
3
4 a_aja = buah_a - buah_b
5 print(a_aja)
6
7 b_aja = buah_b - buah_a
8 print(b_aja)
```

Output :

```
{'mangga', 'apel', 'durian'}
{'anggur', 'alpukat', 'belimbing'}
```

4) Operator Symmetric Difference = menghasilkan Set baru yang merupakan jumlah dari dua Set kecuali irisannya.

Bisa menggunakan operator `^` maupun fungsi `symmetric_difference()`.

```
1 buah_a = {'apel', 'jeruk', 'durian', 'mangga'}
2 buah_b = {'anggur', 'belimbing', 'jeruk', 'alpukat'}
3
4 salah_satu = buah_a ^ buah_b
5 print(salah_satu)
```

Output :

```
{'durian', 'apel', 'belimbing', 'mangga', 'alpukat', 'anggur'}
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
n = int(input("Masukkan jumlah kategori : "))
data_aplikasi = {}

for i in range(n):
    nama_kategori = input(f"Masukkan nama kategori ke-{i+1}: ")
    print(f"Masukkan 5 nama aplikasi di kategori {nama_kategori} : ")
    aplikasi = []
    for j in range(5):
        nama_aplikasi = input(f>Nama aplikasi ke-{j+1}: ")
        aplikasi.append(nama_aplikasi)

    data_aplikasi[nama_kategori] = aplikasi

jumlah_kemunculan = {}
for kategori, daftar_aplikasi in data_aplikasi.items():
    for app in daftar_aplikasi:
        if app in jumlah_kemunculan:
            jumlah_kemunculan[app] += 1
        else:
            jumlah_kemunculan[app] = 1

aplikasi_unik = []
```

```

for app, jumlah in jumlah_kemunculan.items():
    if jumlah == 1:
        aplikasi_unik.append(app)

if len(aplikasi_unik) > 0:
    hasil = ', '.join(aplikasi_unik)
    print(f"Aplikasi yang hanya muncul di satu kategori :
{hasil}")
else:
    print(f"Aplikasi yang hanya muncul di satu kategori :
TIDAK ADA")

if n > 2:
    aplikasi_dua = []
    for app, jumlah in jumlah_kemunculan.items():
        if jumlah == 2:
            aplikasi_dua.append(app)

    if len(aplikasi_dua) > 0:
        hasil = ', '.join(aplikasi_dua)
        print(f"Aplikasi yang muncul di dua kategori :
{hasil}")
    else:
        print(f"Aplikasi yang muncul di dua kategori : TIDAK
ADA")

```

Penjelasan :

1. Buat variabel bernama **n** yang menyimpan input dari user.
2. Kumpulkan input berupa data nama kategori dan nama aplikasi dari setiap kategori menggunakan perulangan **for**.
3. Buat variabel bernama **jumlah_kemunculan** yang berupa set kosong.
4. Cek apakah aplikasi sudah muncul atau belum, menggunakan perulangan **for**.
5. Buat variabel bernama **aplikasi_unik** yang berupa list kosong.
6. Cek satu per satu data dalam **jumlah_kemunculan**, jika jumlah == 1 maka nama aplikasinya akan dimasukkan ke dalam list. Gabung menggunakan fungsi **.join()**.
7. Jika **n > 2**, maka program akan mencari nama aplikasi yang muncul tepat 2 kali, atau ada di kedua kategori.

Output :

```
Masukkan jumlah kategori : 2
Masukkan nama kategori ke-1: Finance
Masukkan 5 nama aplikasi di kategori Finance :
Nama aplikasi ke-1: RTI Saham
Nama aplikasi ke-2: Mirae
Nama aplikasi ke-3: IPOT
Nama aplikasi ke-4: Poems
Nama aplikasi ke-5: Calculator
Masukkan nama kategori ke-2: Utilities
Masukkan 5 nama aplikasi di kategori Utilities :
Nama aplikasi ke-1: Photos
Nama aplikasi ke-2: Weather
Nama aplikasi ke-3: Calculator
Nama aplikasi ke-4: Camera
Nama aplikasi ke-5: Notes
Aplikasi yang hanya muncul di satu kategori : RTI Saham, Mirae,
IPOT, Poems, Photos, Weather, Camera, Notes
```

SOAL 2

Source Code :

```
data_list = ['Valine', 'Ceii', 'Ciaciol', 'Virjie']
print(f"List awal = {data_list}")

list_set = set(data_list)
print(f"List ke Set = {list_set}")

set_list = list(list_set)
print(f"Set ke List = {set_list}")

print()

data_tuple = ('Valine', 'Ceii', 'Ciaciol', 'Virjie')
print(f"Tuple awal = {data_tuple}")

tuple_set = set(data_tuple)
print(f"Tuple ke Set = {tuple_set}")

set_tuple = tuple(tuple_set)
print(f"Set ke Tuple = {set_tuple}")
```

Penjelasan :

1. Buat variabel bernama **data_list** yang menyimpan data tipe list.
2. Untuk mengubahnya ke dalam tipe data set, bisa menggunakan fungsi **set()**.
3. Untuk mengubahnya ke dalam tipe data list, bisa menggunakan fungsi **list()**.
4. Buat variabel bernama **data_tuple** yang menyimpan data tipe tuple.
5. Untuk mengubahnya ke dalam tipe data set, bisa menggunakan fungsi **set()**.
6. Untuk mengubahnya ke dalam tipe data tuple, bisa menggunakan fungsi **tuple()**.
7. Gunakan fungsi **print()** untuk mengeluarkan outputnya.

Output :

```
List awal = ['Valine', 'Cei', 'Ciaciol', 'Virjie']
List ke Set = {'Cei', 'Valine', 'Ciaciol', 'Virjie'}
Set ke List = ['Cei', 'Valine', 'Ciaciol', 'Virjie']

Tuple awal = ('Valine', 'Cei', 'Ciaciol', 'Virjie')
Tuple ke Set = {'Cei', 'Valine', 'Ciaciol', 'Virjie'}
Set ke Tuple = ('Cei', 'Valine', 'Ciaciol', 'Virjie')
```

SOAL 3

Source Code :

```
file1 = input("File pertama : ")
file2 = input("File kedua : ")

try:
    with open(file1) as f1:
        isi_f1 = set(f1.read().lower().split())
    with open(file2) as f2:
        isi_f2 = set(f2.read().lower().split())
    kata_sama = isi_f1 & isi_f2

    if len(kata_sama) > 0 :
        urut = sorted(kata_sama)
        hasil = ", ".join(urut)
        print(f"Kata yang sama di kedua file : \n{hasil}")
    else:
        print("Tidak ada kata yang sama")

except FileNotFoundError:
    print("File tidak ditemukan!")
```

Penjelasan :

1. Buat variabel bernama **file1** dan **file2** yang akan menyimpan input nama file dari user.
2. Gunakan **try-except** untuk menangani error input dari user.
3. Buka isi dari file1 dan file2 lalu simpan dalam variabel **isi_f1** dan **isi_f2**.
4. Gunakan fungsi **read()**, **lower()**, dan **split()** untuk mengubah semua menjadi huruf kecil dan memisahkan berdasarkan spasinya.
5. Lalu ubah menjadi tipe data Set menggunakan fungsi **set()**.
6. Buat variabel bernama **kata_sama** yang akan menyimpan irisan dari **isi_f1** dan **isi_f2**.
7. Jika panjang kata yang sama lebih dari 0, maka urutkan menggunakan fungsi **sorted()** dan tampilkan kata yang sama.
8. Jika tidak ada, maka keluarkan output "Tidak ada kata yang sama."

Output :

```
File pertama : teks1.txt
File kedua : teks2.txt
Kata yang sama di kedua file :
aktivitas, dan, ini, jantung, juga, karena, kesehatan, lari,
memilih, menjaga, olahraga, oleh, tidak, untuk, waktu, yang
```